

s2.04 — HSM-backed key management and signing service with rotation

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Version pinning, signing modes, and rotation — IC3 baseline](#)
- [Lifecycle, deletion, and HSM reconciliation — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: JD fit and preparation addition, recorded in [the bank](#).
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce OCI Vault's implementation or an interviewer's accepted answer.
 - Where OCI behavior is cited it is labeled "documented OCI behavior" with a link.
- Why this is hard
 - Naive: private keys encrypted at rest in a database, a `sign()` function in the API process, and a cron job that overwrites the key yearly.
 - First break: any process that ever holds plaintext key bytes is a compromise story, and overwriting the key in place breaks every verifier and every ciphertext that still depends on the old material.
 - Second break: a signing retry silently resolves an alias to a newer version, a database rollback "undoes" a key the HSM already generated, and one administrator deletes a key that a cold backup still needs.
 - Minimum primitives
 - Immutable HSM-backed key versions behind a stable logical key
 - Alias resolution pinned before the HSM call
 - A lifecycle state machine with a recovery window
 - A durable operation identity that survives the database and HSM boundary.
- 🗣️ "The HSM makes the key unstealable; the service design is about making key versions immutable, resolution pinned, and deletion slow."
- Opening script
 - "I will treat this as a control plane for key lifecycle plus a separate cryptographic data plane that signs and unwraps against an HSM pool."
 - "First I confirm who the callers are, which algorithms and modes we support, and how long a deleted key must remain recoverable."

- “Then I size the HSM pool from signing latency and the certificate authority's issuance load, and define an API where every mutation is a work request.”
- “The deep dive is alias-to-version pinning on the signing path and reconciling key creation across the database and the HSM, because a SQL rollback never undoes an HSM effect.”
- “I will close with failure recovery, separation of duties for deletion, and where I would change the design under stronger requirements.”

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Callers, algorithms, deletion window, primary invariant
Estimates and contract	11:00–17:00	HSM pool size from latency and load; API semantics
Architecture	17:00–30:00	Diagram, control/data plane split, one signing trace
Deep dive	30:00–45:00	Version pinning, RAW vs DIGEST, lifecycle and reconciliation
Operations and tradeoffs	45:00–50:00	Failure table, deletion quorum, tradeoff, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Who calls sign?	The certificate authority issuer (s2.01) plus tenant services signing tokens and artifacts	The CA is low-QPS but correctness-critical; tenant signing sets the throughput ceiling
Symmetric too, or signing only?	Both: asymmetric sign/verify plus envelope encryption (DEK under a KEK)	Envelope path is the high-QPS sibling; it shares versions and states, not latency budget
Key custody?	HSM protection; material never exportable; SOFTWARE protection out of scope	Every operation is an HSM call; the database stores handles, never plaintext key bytes
How is rotation consumed?	Callers reference a logical key or alias; verifiers use exported public keys	Rotation creates a new version; old versions stay usable for verify and decrypt
Deletion recoverability?	Waiting period 7–30 d, default 30 d, cancellable	Forces PENDING_DELETION as a real state with a recovery path and quorum approval
Multi-tenant?	Yes; tenancy and compartment scoped; shared HSM pool with per-tenant logical isolation	Per-tenant quotas and audit; dedicated HSM partitions become an IC4 option
Regional or global?	Regional service; cross-region only via vendor-protected replication	Keeps one metadata authority per region; DR is a separate prompt (s2.06)

- Functional scope
 - Create a logical key with algorithm, purpose, and protection mode.
 - Create immutable versions on rotation
 - Keep prior versions for verify and decrypt.
 - Sign a RAW message or a precomputed DIGEST
 - Return the signature and the exact version used.
 - Export public keys for offline verification
 - Never export private material.
 - Manage aliases that point at one version and can be re-pointed atomically.
 - Disable, schedule deletion, cancel deletion, and delete with a recovery window.
 - Generate and unwrap data encryption keys for envelope encryption.

- Audit every operation with actor, key version, policy decision, and outcome.

Requirement	Target	Enforced by
Sign latency	p99 < 100 ms including authorization; HSM step 10–50 ms (assumption)	Stateless crypto fleet; local version cache; HSM session pool
Sign availability (data plane)	99.99% per region	Crypto fleet across 3 fault domains; last-known-good version cache
Control-plane availability	99.9% per region	Work requests; replicated metadata store
Key creation / rotation	p99 < 30 s to ENABLED as an async operation	202 work request; lifecycle worker with reconciliation
Disable propagation	< 60 s to every signer under normal operation	Cache TTL 60 s plus push invalidation
Deletion recovery window	7–30 d, default 30 d	PENDING_DELETION state with cancel; sweeper honors delete_after
Per-tenant sign quota	100 sign/s default, 429 with Retry-After above	Token bucket per (tenant, key) at the crypto API
Audit completeness	100% of operations, append-only, 7 y retention	Outbox to append-only log; write failure pages immediately
Key custody	0 plaintext private-key bytes outside the HSM boundary	HSM-generated material; handles only in metadata

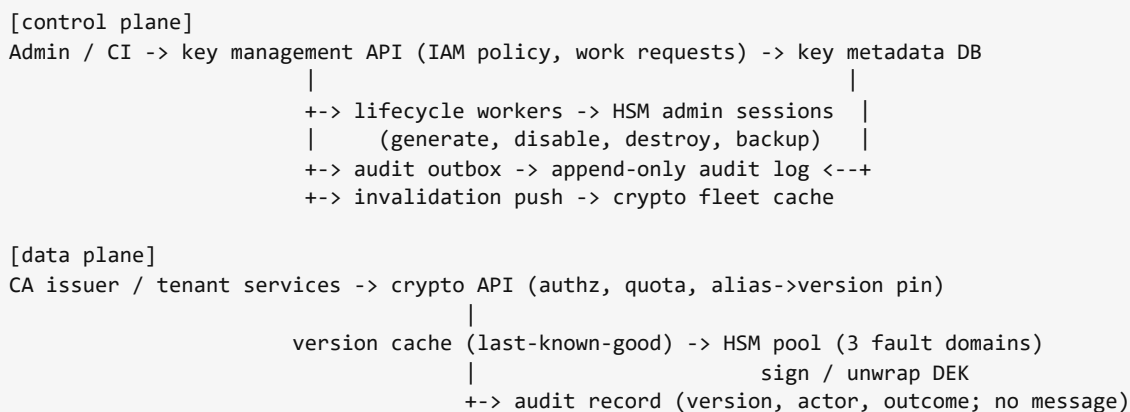
- Invariants
 - A key version's material and algorithm never change after ENABLED
 - Rotation only adds versions.
 - A signing request resolves alias to version before the HSM call
 - The response names that version
 - A retry cannot resolve to a newer version.
 - DISABLED fences new sign and encrypt while verify with exported public keys continues.
 - No version reaches DELETED without passing PENDING_DELETION for at least the minimum window.
 - Every logical operation has a durable audit intent and one committed logical result.
 - Physical HSM attempts have distinct audit events and may repeat after an uncertain response.
 - The database rows are the authority for lifecycle state
 - The HSM is the authority for material existence
 - Reconciliation resolves disagreement.
- Non-goals
 - Bring-your-own-key import and external key stores.
 - Certificate issuance policy (owned by s2.01).
 - Cross-region active-active key metadata.

Capacity and consequence — IC3 baseline

- Signing load
 - CA issuance from s2.01: about 16k renewals/day, 0.2/s average, bursts near 50/s when a CA rotates and fans out.
 - Tenant signing (tokens, artifacts) assumed at 400/s regional peak.
 - Regional peak on the order of 500 sign/s
 - This is an assumption, not a measured number.
 - Decision: the CA alone never sizes the pool
 - Tenant signing does, so per-tenant quotas protect the CA's share.
- HSM pool from latency
 - Little's law: $500/s \times 0.03 s \approx 15$ signatures in flight at a 30 ms HSM step.

- Fifteen concurrent requests estimate occupancy, not sustainable signature throughput.
- Benchmark algorithm mix, session limits, utilization, and p99 before sizing the pool.
- Decision: place the proposed three HSM members in supported independent failure domains.
- Verify that the remaining measured capacity handles the target mix after the largest planned failure.
- Decision: rising HSM latency consumes concurrency headroom and requires a new capacity measurement.
- Envelope encryption
 - Assumed 5,000/s DEK unwrap peak
 - Symmetric unwrap inside the HSM is on the order of 1-5 ms.
 - In flight is about $5,000 \times 0.003 \approx 15$, comparable to signing despite $10\times$ the rate.
 - Decision: the DEK path shares the HSM pool but gets its own session budget so a decrypt storm cannot starve CA signing.
 - Decision: callers cache DEKs per object or bounded batch
 - One HSM call per object, never per byte.
- Metadata
 - 10k tenants $\times$ 50 keys $\times$ 10 versions = 5M version rows at about 300 B each, on the order of 1.5 GB.
 - Decision: one regional relational store with read replicas
 - The crypto fleet reads from a local cache, so metadata reads are not on the signing path.
- Audit
 - About 5,500 operations/s $\times$ 500 B $\approx$ 2.75 MB/s, roughly 240 GB/day, hundreds of TB over 7 y before compression.
 - Decision: audit is a partitioned append-only log tiered to object storage, not rows in the metadata database.
 - Decision: audit stores key version, operation identity, and outcome
 - Never message bytes, so volume stays bounded and no payload leaks.

Architecture and flow — IC3 baseline



- Control plane owns key, version, and alias lifecycle.
 - Mechanism bought: transactional state transitions with optimistic versioning and work requests for every mutation.
 - It talks to HSM admin sessions only through lifecycle workers, never inline in a request.
 - Cost: creation and rotation are asynchronous, so the API must expose operation status and the database and HSM can disagree after a crash.
- Crypto data plane owns sign, verify-helper, and envelope operations.
 - Mechanism bought: a stateless fleet that resolves (tenant, key, version) to an HSM handle from a local cache and holds pooled HSM sessions.
 - Static stability: signers keep serving on the last-known-good cache when the metadata store or control plane is down.

- Cost: a disable or deletion issued during a control-plane outage is not visible to signers until the cache refreshes
 - The design bounds that staleness rather than eliminating it.
 - HSM pool owns key material.
 - Mechanism bought: material generated and used inside the boundary
 - The service only holds handles and public keys.
 - Cost: throughput is bounded by sessions and per-operation latency
 - Backup and replication must use vendor-protected mechanisms, not metadata copies.
 - Audit log owns the evidence trail.
 - Mechanism bought: append-only records written through an outbox in the same transaction as the state change.
 - Cost: audit write failure must page and must not silently drop
 - Sign audit is asynchronous so a sink outage degrades evidence, not availability.
 - Documented OCI behavior for orientation
 - OCI Vault exposes a management endpoint for key lifecycle and a separate cryptographic endpoint for encrypt, decrypt, and sign
 - That is the control/data split above.
 - Keys carry versions and a protection mode of `HSM` or `SOFTWARE`
 - Rotation creates a new version.
 - See [OCI Vault key overview](#).
- Authenticate the caller and authorize `use keys` on the target compartment and key.
- Check the per-tenant token bucket; reject with `429` and `Retry-After` when exhausted.
- Resolve alias or logical key to one `key_version` from the local cache and pin it in the intent record.
- Verify version state is `ENABLED` and purpose permits `SIGN` with the requested algorithm.
- Validate mode: `DIGEST` length must match the hash algorithm; `RAW` length must be under the HSM limit.
- Acquire an HSM session from the pool and call sign with the pinned handle.
- Return `{signature, keyId, keyVersion, algorithm}`; store the response under the idempotency key when present.
- Emit an audit record with actor, key version, algorithm, outcome, and trace id; no message or digest bytes.

API and data model — IC3 baseline

Surface	Contract	Conflict/error behavior
POST <code>/v1/keys</code>	<code>{name, algorithm, purpose, protection: 'HSM'}</code> ; returns <code>202 {keyId, operationId}</code>	<code>I</code> ; same key plus different payload returns <code>409</code> ; unsupported algorithm returns <code>422</code>
POST <code>/v1/keys/{id}/versions</code>	<code>{reason}</code> ; returns <code>202 {keyVersion, operationId}</code>	<code>I</code> ; rotation adds material and never deletes prior versions; <code>409</code> if key not <code>ENABLED</code>
GET <code>/v1/keys/{id}/versions/{v}/public-key</code>	SPKI public key plus fingerprint	Asymmetric only; <code>404</code> for symmetric keys; no private-key path exists
POST <code>/v1/keys/{id}/signatures</code>	<code>{keyVersion?, alias?, messageType: 'RAW' or 'DIGEST', hashAlgorithm, message, signingAlgorithm}</code> ; returns <code>200 {signature, keyVersion, algorithm}</code>	<code>I</code> optional; <code>409 KEY_VERSION_DISABLED</code> if pinned version is not <code>ENABLED</code> ; <code>422</code> on digest length mismatch

Surface	Contract	Conflict/error behavior
PUT /v1/key-aliases/{alias}	{keyId, keyVersion}; returns alias with ETag	V; 412 on stale ETag; re-pointing never changes existing signatures
PATCH /v1/keys/{id}/versions/{v}	{state: 'DISABLED' or 'ENABLED'}; returns version with ETag	V; 409 if version is PENDING_DELETION or DELETED
POST /v1/keys/{id}/versions/{v}/deletion-requests	{deleteAfter, reason}; returns 202 operation	I; 422 if window outside 7-30 d; requires second approver before scheduling
POST /v1/keys/{id}/versions/{v}:cancelDeletion	Returns 200 version	409 if already DELETED; audited with approver identity
POST /v1/keys/{id}/data-keys	{keySpec: 'AES_256', context}; returns plaintext DEK plus wrapped DEK	Never idempotent by design; 409 if key not ENABLED
GET /v1/operations/{operationId}	Operation state and result reference	Authoritative answer after an ambiguous create or rotate timeout

State	Key	Fields/invariant
Logical key	(tenant_id, key_id)	Name, purpose, algorithm, current version pointer, row version; name unique per tenant
Key version	(tenant_id, key_id, key_version)	hsm_handle, public key reference, fingerprint, state, created_at, delete_after; immutable material
Alias	(tenant_id, alias)	Target (key_id, key_version), row version; ETag on every update
Sign intent	(tenant_id, caller_id, 'sign', intent_key)	Request hash, pinned key_version, stored response; retention at least the retry horizon
Lifecycle operation	(tenant_id, operation_id)	Type, target version, state, HSM label, approver ids; drives reconciliation
Deletion request	(tenant_id, key_id, key_version)	Requester, approver, delete_after, state; two distinct principals required
Audit event	(tenant_id, event_id)	Actor, action, key version, algorithm, outcome, trace id; redacted details only

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - Key material is never relational
 - hsm_handle and hsm_label name HSM objects.
 - The sign intent row lives in the shared api_intents table with operation = 'sign'.

```

CREATE TABLE crypto_keys (
  tenant_id text NOT NULL, key_id text NOT NULL, name text NOT NULL,
  purpose text NOT NULL, algorithm text NOT NULL,
  current_version bigint, -- used when a sign request pins neither version nor
alias
  version bigint NOT NULL DEFAULT 1, -- row version behind the ETag
  PRIMARY KEY (tenant_id, key_id),
  UNIQUE (tenant_id, name),
  CHECK (purpose IN ('SIGN', 'ENCRYPT', 'WRAP'))
);
CREATE TABLE key_operations (
  tenant_id text NOT NULL, operation_id text NOT NULL, key_id text NOT NULL,
  op_type text NOT NULL, key_version bigint, state text NOT NULL,
  hsm_label text NOT NULL, -- equals operation_id at generate time; the reconciler
inventories by it
  requester_id text NOT NULL, approver_id text,
  accepted_at timestamptz NOT NULL, finished_at timestamptz,
  PRIMARY KEY (tenant_id, operation_id),
  FOREIGN KEY (tenant_id, key_id) REFERENCES crypto_keys,
  CHECK (op_type IN ('CREATE_KEY', 'CREATE_VERSION', 'SCHEDULE_DELETION', 'DESTROY')),
  CHECK (state IN ('ACCEPTED', 'RUNNING', 'SUCCEEDED', 'FAILED'))
);
CREATE INDEX key_operations_open ON key_operations (accepted_at)
  WHERE state IN ('ACCEPTED', 'RUNNING');
CREATE TABLE crypto_key_versions (
  tenant_id text NOT NULL, key_id text NOT NULL, key_version bigint NOT NULL,
  hsm_handle text, -- HSM handle, never key material; NULL only while
CREATING
  public_key_ref text, fingerprint text,
  state text NOT NULL, operation_id text NOT NULL, -- the create operation; recovery scans CREATING rows by
it
  created_at timestamptz NOT NULL, delete_after timestamptz,
  PRIMARY KEY (tenant_id, key_id, key_version),
  FOREIGN KEY (tenant_id, key_id) REFERENCES crypto_keys,
  FOREIGN KEY (tenant_id, operation_id) REFERENCES key_operations,
  CHECK (state IN ('CREATING', 'ENABLED', 'DISABLED', 'PENDING_DELETION', 'DELETED')),
  CHECK (state = 'CREATING' OR hsm_handle IS NOT NULL),
  CHECK (state <> 'PENDING_DELETION' OR delete_after IS NOT NULL)
);
CREATE INDEX key_versions_deletion_sweep ON crypto_key_versions (delete_after)
  WHERE state = 'PENDING_DELETION';
CREATE INDEX key_versions_stuck_creating ON crypto_key_versions (created_at)
  WHERE state = 'CREATING';
CREATE TABLE key_aliases (
  tenant_id text NOT NULL, alias text NOT NULL, key_id text NOT NULL,
  key_version bigint NOT NULL, version bigint NOT NULL DEFAULT 1,
  PRIMARY KEY (tenant_id, alias),
  FOREIGN KEY (tenant_id, key_id, key_version) REFERENCES crypto_key_versions
);
CREATE TABLE deletion_requests (
  tenant_id text NOT NULL, key_id text NOT NULL, key_version bigint NOT NULL,
  requester_id text NOT NULL, approver_id text, -- two distinct principals before SCHEDULED
delete_after timestamptz NOT NULL, reason text NOT NULL,
  state text NOT NULL, requested_at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, key_id, key_version),
  FOREIGN KEY (tenant_id, key_id, key_version) REFERENCES crypto_key_versions,
  CHECK (state IN ('PENDING_APPROVAL', 'SCHEDULED', 'CANCELED', 'EXECUTED')),
  CHECK (approver_id IS NULL OR approver_id <> requester_id),
  CHECK (state <> 'SCHEDULED' OR approver_id IS NOT NULL),
  CHECK (delete_after >= requested_at + interval '7 days'
  AND delete_after <= requested_at + interval '30 days')
);
CREATE TABLE key_audit_events (
  tenant_id text NOT NULL, event_id text NOT NULL, at timestamptz NOT NULL,
  actor_id text NOT NULL, action text NOT NULL,
  key_id text, key_version bigint, algorithm text,
  outcome text NOT NULL, trace_id text NOT NULL,
  details jsonb NOT NULL DEFAULT '{}', -- redacted; never message bytes or digests

```

```
PRIMARY KEY (tenant_id, event_id)
);
CREATE INDEX key_audit_by_key ON key_audit_events (tenant_id, key_id, key_version, at DESC);
```

- What the DDL does not capture
 - Private key bytes stay inside the HSM; a metadata backup restores handles and states only.
 - Sign intents use the shared `api_intents` table keyed `(tenant_id, caller_id, 'sign', intent_key)`; the pinned `key_version` sits in the stored response body.
 - Plaintext data keys are returned once and never persisted.
 - Deviation from the companion
 - `current_version` on the key row, `operation_id` on the version row, and the `key_operations`, `deletion_requests`, and `key_audit_events` tables are additions.
 - The version state machine and the 7-30 d deletion window become `CHECK` constraints.
- Proposed service conventions
 - `keyVersion` and `alias` are mutually exclusive in a sign request
 - Omitting both uses the key's current version and the response still names the version.
 - `DIGEST` mode carries a hash already computed by the caller
 - The service never hashes it again.
 - `RAW` mode hashes inside the boundary and is capped at an HSM-specific message size, on the order of 4 KB.
 - Signing algorithm identifiers are explicit (`ECDSA_SHA_256`, `RSA_PSS_SHA_256`, `RSA_PKCS1_SHA_256`)
 - The caller cannot request a hash that the key's purpose forbids.
 - `I` and `V` follow the [shared conventions](#)
 - The reference DDL above adapts the [companion](#) schema to this file's vocabulary; the companion stays the shared baseline.

Version pinning, signing modes, and rotation — IC3 baseline

- Alias resolution is pinned before the HSM call
 - The crypto API reads the alias target once, writes the pinned `key_version` into the intent record, then calls the HSM.
 - A retry with the same idempotency key replays the stored response or re-signs with the pinned version, never with a newer alias target.
 - Without pinning, a rotation between attempt one and attempt two yields two valid signatures under different versions and a CA that recorded the wrong issuer version.
- Deterministic versus randomized signatures
 - `RSA_PKCS1` and deterministic `ECDSA` (RFC 6979 style) return identical bytes on re-sign.
 - `RSA_PSS` and standard `ECDSA` use fresh randomness, so a retry produces a different but equally valid signature.
 - Consequence: a caller that needs byte-identical replay must send an idempotency key so the service stores and replays the first signature.
 - Consequence: a caller that only needs one valid signature recorded (the CA) treats any returned signature as final once its own database commit succeeds.
- `RAW` versus `DIGEST`
 - `DIGEST` mode: the caller sends `SHA-256(tbsCertificate)`
 - The HSM applies the signature primitive to that digest with the correct algorithm identifier encoding.
 - `RAW` mode: the caller sends the message
 - The boundary hashes it first.
 - Trap: sending a digest in `RAW` mode hashes the digest again, so verifiers computing `SHA-256(message)` reject the signature.
 - Trap: a `DIGEST` request whose length does not match `hashAlgorithm` is rejected with 422, not signed.

- The CA may prefer DIGEST mode to control hashing and request size.
- TBS certificate bytes are public, and not every certificate exceeds a RAW size cap.
- Match the digest algorithm to the chosen signature suite rather than assuming SHA-256 for every key.
- Rotation with overlap

key K: current -> v3 (ENABLED)	new sign/encrypt use v3
v2 (ENABLED)	verify/decrypt still valid; disable after adoption check
v1 (DISABLED)	verify with exported public key still works offline
alias "ca-issuer" -> (K, v3)	re-pointed under ETag after v3 reaches ENABLED

- Rotation never re-signs old signatures or re-encrypts old ciphertext
- It only changes what new operations use.
- Retirement requires inventory of dependent ciphertext and retained backups, plus any rewrap plan.
- Fourteen days without decrypt calls does not prove old encrypted data is no longer needed.
- For a CA signing key, the old version stays `ENABLED` until the last certificate it issued expires or the CRL it must sign is retired, otherwise CRL signing breaks.
- Documented OCI behavior: rotation creates a new key version and prior versions remain available for decryption
 - See [OCI Vault key overview](https://docs.oracle.com/en-us/iaas/Content/KeyManagement/Concepts/keyoverview.htm).

- Signing flow
 - Claim and pin the operation atomically before using the HSM.
 - Commit the selected result before returning a replayable success.

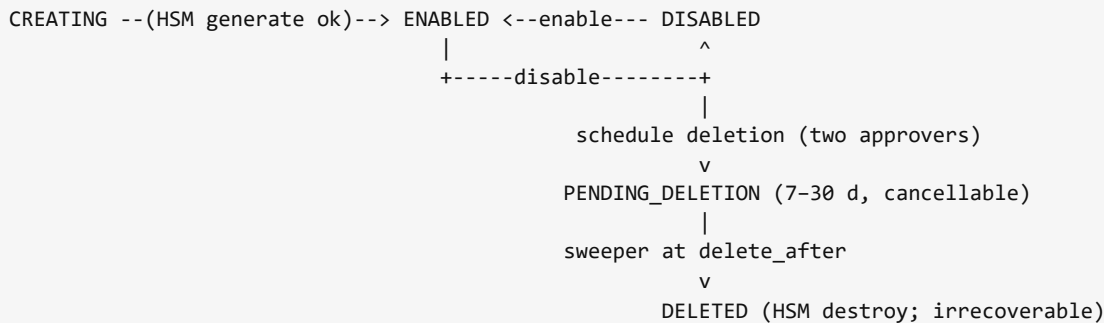
```

SIGN(tenant, caller, request):
  authenticate and authorize exact action and key scope
  transaction:
    intent = claim_or_read(tenant, caller, action, token, canonical_hash)
    if committed: return recorded response after current replay authorization
    require no live competing attempt
    resolve and persist immutable key version and algorithm parameters
    increment attempt fence and record audit attempt intent
  require current signer ownership and unexpired key-state/policy authorization
  validate RAW/DIGEST mode and algorithm-specific digest length
  signature = HSM.sign(pinned_version, validated_parameters)
  transaction:
    require current attempt fence and permitted result-release state
    commit chosen signature, logical result, audit, and outbox
  return committed signature and pinned version

```

- Result ambiguity
 - A timeout before the result commit may require another physical signature attempt.
 - Byte-identical replay is guaranteed only for a durably selected committed response within retention.
 - Best-effort response storage cannot support that promise.
- Disable boundary
 - A 60 s cached authorization lease gives a bounded stale-use window, not immediate disable.
 - Immediate prevention of new signing requires an ordered barrier enforced by the signing broker.
 - PKCS#11 key handles do not enforce an application epoch by themselves.
 - A signature already produced cannot be undone by a later database check.

Lifecycle, deletion, and HSM reconciliation — IC3 baseline



- State semantics
 - **CREATING**: metadata row exists, HSM material may or may not exist yet.
 - **ENABLED**: sign, encrypt, decrypt, and data-key generation permitted.
 - **DISABLED**: sign and encrypt fenced
 - Decrypt is also fenced in this proposed design
 - Verify with exported public keys continues because it never touches the service.
 - **PENDING_DELETION**: all cryptographic use fenced
 - `cancelDeletion` returns to **DISABLED**.
 - **DELETED** means the selected key material is destroyed under the provider contract.
 - Ciphertext is unrecoverable only if no usable DEK, alternative wrapper, or permitted recoverable backup remains.
 - Documented OCI behavior: a scheduled key deletion has a waiting period between 7 and 30 days during which the deletion can be cancelled
 - See [OCI Vault key overview](#).
- Separation of duties and quorum
 - Key administrators manage lifecycle
 - Key users call sign and decrypt
 - Auditors read the log
 - No principal holds two of these roles on the same compartment.
 - Deletion requires a requester and a distinct approver
 - The request record stores both identities and the transition is refused until the second approval lands.
 - Alias re-pointing on a CA signing key requires the same two-person rule, because it changes which version signs certificates.
 - Documented OCI behavior: IAM policy statements scope who can manage or use keys per compartment
 - See [IAM policies](#).
- Reconciliation across the database and the HSM
 - Create version: insert row **CREATING** with `operation_id` in one transaction
 - Call HSM generate with label = `operation_id`
 - Update row to **ENABLED** with the returned handle.
 - Crash after HSM generate but before the update: the row stays **CREATING**, the HSM holds unlabeled-in-database material.
 - Recovery inventories objects by operation identity and verifies their attributes.
 - A label is not a uniqueness guarantee unless the provider enforces it.
 - Quarantine duplicates and uncertain generation instead of blindly creating another key.
 - A SQL rollback does not undo an HSM effect
 - The worker must never assume a failed transaction left the HSM clean.
 - Sign retries reuse the pinned intent and key version.
 - HSM auditing and counters may record multiple attempts
 - Randomized signatures may differ.

- Destroy: mark `DELETED` only after the HSM confirms destruction
 - A confirmed destroy followed by a lost update is repaired by the reconciler, which finds no HSM object for the handle.
- Backup and replication
 - Key material leaves the HSM only inside vendor-protected backups or cluster replication wrapped under HSM-domain keys.
 - Metadata backups reproduce handles and states, never private keys
 - Restoring metadata without the matching HSM backup yields rows that point at nothing.
 - Backup and restore are separate authorizations from lifecycle management and are audited as their own operations.
 - [OCI backup documentation](#) defines supported vault/key backup and restore paths.
 - Select the vault type and supported target region before promising key recoverability.
 - A successful protected restore drill is the acceptance criterion.
- Envelope encryption as the sibling use case
 - `generateDataKey` returns a plaintext `DEK` and the same `DEK` wrapped under the `KEK` version
 - The caller encrypts locally and stores only the wrapped `DEK` with the ciphertext.
 - Decrypt unwraps the `DEK` inside the HSM under whichever version wrapped it
 - The wrapped blob carries the version identifier.
 - Rotating the `KEK` does not re-wrap old `DEK`s
 - A re-wrap job exists only if policy requires retiring old material.
 - Deleting an old `KEK` can make ciphertext whose only `DEK` wrapper uses that version unrecoverable.
 - Cached plaintext `DEK`s and independent wrapped copies are not retroactively erased.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
One HSM fails	One fault domain's sessions	Session errors, health probe	Pool routes to the remaining two; latency rises	Replace unit; material restored via vendor cluster sync or backup
All HSMs unreachable	New HSM-dependent operations	Session failures	Sign/decrypt fail closed; offline public-key verification continues	Restore HSM access; cached-key TLS continues only while its own material and status remain valid
Metadata store unavailable	Lifecycle mutations and signer freshness	Quorum and cache-age alarms	Only operations with an unexpired authorization/state lease may continue	Restore authority; expired leases fail closed; reconcile uncertain attempts
Sign response lost	One request	Client timeout	Retry with same idempotency key against pinned version	Replay stored response or re-sign with the same version
Crash between HSM generate and row update	One key version	Stuck <code>CREATING</code> intent	Hold uncertain generation	Inventory and verify candidates; adopt one proven match or quarantine ambiguity
Digest sent in RAW mode	One caller	Signature verification mismatch	The service may validly sign the digest bytes as a message	Client must select the correct mode; byte length alone cannot identify intent
Old <code>KEK</code> version deleted with live <code>DEK</code> s	Every object wrapped under it	Decrypt failures after <code>delete_after</code>	None; ciphertext is unrecoverable	Prevention only: adoption check and quorum before scheduling
Audit sink unavailable	Audit delivery backlog	Oldest outbox age and bytes	Continue only while durable local audit capacity remains within its bound	Drain outbox; reject new HSM work before durable audit admission is exhausted

Failure	Blast radius	Detection	Degraded behavior	Recovery
Tenant exceeds quota	One tenant's sign calls	429 rate per tenant	Retry-After with jitter; CA share protected	Raise quota or move tenant to dedicated partition

- Observe
 - HSM sign latency p50/p99 per unit and session-pool saturation.
 - Operations per key version, which feeds adoption checks before disable or delete.
 - Oldest `CREATING` row and oldest unfinished operation.
 - Cache staleness: seconds since the last successful metadata refresh on each signer.
 - 429 rate per tenant and audit outbox depth.
 - Any sign call against a root or CA key outside a scheduled window pages.
- Retry discipline
 - Retry 503, HSM session timeouts, and metadata transient errors with exponential backoff and jitter.
 - Never retry 403, 409 `KEY_VERSION_DISABLED`, 422, or a policy denial
 - Those need a corrective action.
 - Reuse the idempotency key on ambiguous timeouts
 - A new intent means a new key.
 - Bound total elapsed time so a CA renewal that cannot sign fails fast and reschedules rather than pinning HSM sessions.
- Tenant isolation
 - Keys, versions, aliases, and operations are compartment-scoped
 - Policy evaluation happens before any cache lookup.
 - Quotas cap sign rate and active keys per tenant
 - The CA's own tenancy has a reserved session budget.
 - Cross-tenant key ids return 404, not 403, to avoid existence leaks.
 - Audit records carry tenant and caller identity
 - A tenant can read only its own compartment's events.
- Rollout and rollback safety
 - Schema changes follow expand, migrate, contract
 - A new state value is unknown to old signers, so signers treat unknown states as fenced.
 - Canary the crypto fleet per fault domain
 - In-flight sign calls survive a deploy because state lives in the metadata store and the HSM, not in the process.
 - A new HSM firmware or driver rolls one unit at a time with the other two carrying load.
 - Rollback of a lifecycle worker must tolerate `CREATING` rows labeled by the newer worker
 - Label format is versioned.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Software keys encrypted at rest	Private keys in the database under a software KEK	Cheap; unlimited throughput	Any process compromise exports keys; encrypted-at-rest is not custody	Dev and test only
HSM-resident keys	Every version occupies an HSM slot	Simplest custody story	Slot count bounds key count; replication by vendor only	Few high-value keys such as CA signers
HSM-wrapped key blobs	Material wrapped under an HSM domain key and stored as ciphertext in metadata; unwrapped inside the HSM per operation	Unbounded key count; any HSM in the domain serves any key	Domain key distribution is the crown jewel; adds unwrap latency	Baseline multi-tenant KMS

Option	What it is	Strength	Weakness	Choose when
Dedicated HSM partition per tenant	Physically or logically separate partition	Strongest isolation; independent quotas	Cost and partition management	Regulated tenants; documented OCI offering is the virtual private vault
Delegated short-lived signers	HSM signs a short-lived sub-key that a local process uses	Moves HSM off the hot path	Sub-key exposure bounded by lifetime, not zero	Per-pod certificates or high-volume token signing

- IC4 stretch — stronger custody requirement
 - Require that every use of a CA signing key be attributable to a scheduled ceremony or an issuance operation id.
 - Bind the sign request to the operation id in the intent record and refuse sign calls on ceremony-class keys without one.
 - Alert on any sign against a root key
 - The root exists to sign subordinates, so any other use is an incident.
- IC4 stretch — 10× load
 - 5,000 sign/s puts about 150 signatures in flight
 - Session arithmetic is an occupancy estimate, not a valid HSM count.
 - Choose the count from measured throughput and tail latency after the largest planned domain loss.
 - If the vendor supports securely shared wrapped-key blobs, evaluate that path to reduce per-device key-slot pressure.
 - Verify the authorized HSM domain and key attributes before assuming another device can use a blob.
 - Move high-volume token signing to delegated short-lived signers only if the trust protocol and custody policy permit it.
 - The HSM signs one delegate per node per day instead of every token.
 - Shard the token bucket and intent store by (tenant, key)
 - The metadata store stays out of the signing path.
- IC4 stretch — cost and blast radius
 - HSM units dominate cost
 - The proposed three-device topology is an assumption, not a universal HSM minimum.
 - Capacity, vendor replication support, and failure-domain placement determine the real minimum.
 - Cell the crypto fleet and metadata per availability domain so a metadata store failure strands one cell's lifecycle, not the region's signing.
 - A shared HSM domain key means a domain compromise is a region-wide event
 - Dedicated partitions convert that into a per-tenant event at a per-tenant price.
- IC4 stretch — migration and rollout
 - Migrating from HSM-resident keys to wrapped blobs happens per version: wrap inside the HSM, store the blob, verify a sign against both paths, then retire the slot.
 - Never migrate a CA signing key during its issuance window
 - Schedule it against the CA rotation calendar in s2.02.
- Weakest assumption
 - 30 ms HSM sign latency with 32 idle sessions
 - A correlated storm of CA renewals plus tenant token signing after a fleet restart can saturate sessions and turn p99 < 100 ms into seconds.
 - Mitigation: per-tenant token buckets with Retry-After, renewal jitter in the CA scheduler, and delegated signers for the highest-volume tenants.
- Over-engineering warning
 - Do not build custom HSM cluster synchronization
 - Use vendor replication and backup and design the reconciler around them.
 - Do not put every application token signature on the HSM

- Delegate short-lived signers and keep the HSM for keys whose compromise is unbounded.
- Do not promise byte-identical signature replay for randomized algorithms without an intent store.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Callers, algorithms, RAW vs DIGEST, deletion window, custody invariant	Names envelope encryption as the sibling path with its own quota and states	Separates the two load profiles before drawing anything
Estimates	Mean latency estimates occupancy; measured throughput sizes devices	Algorithm mix and largest-domain-loss headroom	Does not mistake session count for capacity
Architecture	Control plane, crypto data plane, HSM pool, audit; one signing trace	States the staleness cost of static stability and bounds it	Every component has a mechanism and a price
Deep dive	Alias pinning, immutable versions, lifecycle states, database versus HSM reconciliation	Deterministic versus randomized signatures; label-based adoption; delegation	Reasons across the database and HSM boundary
Operations	Failure table, quota, audit without payloads, two-person deletion	Rollout of unknown states, HSM firmware rolls, cell blast radius	Volunteers separation of duties before being asked
Wrap	Recap and tradeoff	Names the session-saturation assumption first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Why not resolve the alias inside the HSM call and skip pinning?
 - **IC3 baseline**
 - A retry after rotation would sign under a newer version and the caller could not tell which version produced which signature.
 - **IC4 stretch**
 - Pin in a durable intent row so replay and audit agree
 - Expose the version in every response and in the audit record.
- Q2: The signing request timed out.
 - Did the HSM sign?
 - **IC3 baseline**
 - Possibly
 - Retry only under the same immutable intent and permitted pinned key version.
 - Physical attempts can repeat and must remain audited.
 - **IC4 stretch**
 - For randomized algorithms the retry yields different bytes
 - Store the first response under the idempotency key if the caller needs byte-identical replay.
- Q3: A database rollback happened after the HSM generated key material.
 - Is the HSM clean?
 - **IC3 baseline**
 - No
 - The material exists and the row is `CREATING`
 - A reconciler must verify and adopt a proven matching object or quarantine ambiguity.
 - Regenerate only after establishing that no earlier generation effect remains.
 - **IC4 stretch**
 - Label objects with operation identity and verify their attributes.
 - Provider-enforced uniqueness is required before treating the label as an idempotency guarantee.
- Q4: Rotation happened.
 - Is the old ciphertext now unreadable?

- **IC3 baseline**
 - No
 - Rotation adds a version and old versions stay `ENABLED` for decrypt and verify.
- **IC4 stretch**
 - Name the re-wrap job as a separate policy decision and the adoption check that must pass before the old version is disabled.
- Q5: Why does deletion need two people and a month?
 - **IC3 baseline**
 - Deletion can make data unrecoverable when the last usable key path is lost.
 - Inventory wrapped DEKs and protected backups before approving it.
 - The window turns a mistake into a cancellation.
 - **IC4 stretch**
 - Require adoption evidence from audit during the window and block the sweeper while any operation against the version is observed.
- Q6: The metadata store is down.
 - Can the CA still issue?
 - **IC3 baseline**
 - Signers keep serving from the last-known-good cache, so signing continues
 - New keys and rotations wait.
 - **IC4 stretch**
 - State the cost out loud: a disable issued during the outage is invisible until refresh
 - Bound it with cache TTL and push invalidation once the store returns.
- Q7 (premise reversal): Why not store keys encrypted in the database under a software KEK and skip the HSM entirely?
 - **IC3 baseline**
 - Encrypted-at-rest is not custody
 - Any process that decrypts the key can export it, and the KEK itself becomes the unprotected root.
 - **IC4 stretch**
 - Software protection is acceptable only for low-value, short-lived, or high-volume keys, and then it should be a delegated signer whose parent lives in the HSM.
- Q8: Tenant token signing is starving the certificate authority.
 - **IC3 baseline**
 - Per-tenant token buckets with `429` and `Retry-After`
 - A reserved session budget for the CA's tenancy.
 - **IC4 stretch**
 - Move that tenant to delegated short-lived signers or a dedicated partition so its volume never touches the shared pool.
- Validation scenarios
 - Retry after alias re-point
 - Response and audit name the pinned version.
 - Digest submitted in `RAW` mode
 - Verification fails and the mode check catches length mismatch.
 - Crash between HSM generate and `ENABLED` update
 - Reconciler adopts by label.
 - Disable during a control-plane outage
 - Measure propagation delay after recovery.
 - Deletion request without a second approver
 - Transition refused and audited.
 - Cancel deletion on the last day of the window
 - Version returns to `DISABLED`.

- One HSM unit removed under peak load
- Latency stays within budget.
- Audit sink outage
- Sign continues, lifecycle blocks at outbox bound, page fires.

Recall card

- “Logical key stays stable; versions are immutable; rotation only adds.”
- “Pin the version before the HSM call; the response and the audit name it.”
- “DIGEST means sign this hash; RAW means hash then sign; mixing them signs the wrong message.”
- “A SQL rollback never undoes an HSM effect; label the material and reconcile.”
- “Deletion is slow on purpose: two people, a window, and adoption evidence.”
- Wrap-up script
 - “The design is a lifecycle control plane over immutable HSM-backed key versions, with a stateless crypto data plane that signs and unwraps against a three-unit HSM pool.”
 - “The key invariant is that every signing request pins one version before the HSM call, so retries and rotation can never disagree about which key signed.”
 - “The main tradeoff is static stability: signers keep serving from a cached resolution during a metadata outage, which buys availability at the cost of bounded staleness for disables.”
 - “My weakest assumption is HSM session headroom under a correlated renewal and token-signing storm; I would validate quota and delegation under that load first.”

References

- [Ranked question and report provenance.](#)
- [OCI role framing.](#)
- [Detailed API and schema companion.](#)
- [Shared API conventions.](#)
- Reference check: 2026-09-09.
 - Official documentation supports the mechanisms cited next to each link.
 - Latency, load, session, and pool-size values are proposed interview assumptions.
- [OCI Vault key overview](#): management versus cryptographic endpoints, HSM and software protection modes, key versions and rotation, deletion waiting period.
- [OCI IAM policies](#): compartment-scoped manage versus use verbs behind separation of duties.
- [OCI Certificates overview](#): CA private keys held in Vault-backed HSM keys, the consumer of this service.
- [RFC 5280](#): `signatureValue` over `tbsCertificate`, the reason the CA sends a digest.
- [PKI and KMS mechanism reference](#): Part B key states, envelope encryption, `Sign` with `DIGEST`, and the KMS quota ceiling on issuance.
- [Multi-tenant certificate authority](#): the primary caller
 - Issuance latency budget and CA rotation calendar.